

BENCHMARK DE COSTOS BANCARIOS

TRABAJO FINAL DE TÍTULO



OBJETIVOS

Objetivo general:

Construir un benchmark respaldado por una API que calcule el Costo Total Anual (Annual Total Cost) para cuentas vista según un perfil de uso y recomiende el plan de menor costo esperado para el usuario.

Objetivos específicos:

1. Consolidar datos oficiales: ingestión y unificación de CMF y Banco Central (UF) en Parquet.
2. Estandarizar esquema: definir un esquema canónico y reglas de validación para costos mensuales en CLP y valores normalizados en UF.
3. Motor de cálculo: implementar un ATC determinista y generar rankings comparativos.
4. API y visualización: exponer endpoints mínimos y una vista comparativa básica para la toma de decisiones.

JUSTIFICACIÓN

La transparencia financiera permite a consumidores y PYMEs (Pequeñas y medianas empresas) elegir productos bancarios con menor costo total. La CMF publica un simulador con un “costo neto total” mensual por institución y producto, aunque la información está dispersa para comparaciones masivas. Centralizarla, normalizarla con la UF oficial del Banco Central y exponerla mediante un servicio reproducible reduce la asimetría de información y mejora la comparabilidad. En una fase posterior se incorporarán tablas oficiales del SERNAC para ampliar el alcance a tarjetas.

“**SERNAC** es el organismo público chileno que **protege los derechos de los consumidores**. En pocas palabras: **es la institución que vela porque las empresas cumplan la ley y traten bien a los clientes**, recibiendo reclamos, mediando conflictos y fiscalizando prácticas comerciales”

CODIGOS UNESCO

- 1209.03: Análisis de Datos (Cálculo de ATC y benchmarking).
- 1203.96: Bases de Datos (Almacenamiento y estructuración de datos bancarios).
- 1203.88: Inteligencia de Negocio (Generación de valor para la toma de decisiones).
- 1203.93: Cloud Computing (Arquitectura API y despliegue en contenedores).

ALCANCE

En esta entrega:

- Ingesta de CMF simulación promedio de cuentas vista (HTML tabular) y serie UF (BDE o CMF API).
- Limpieza: parseo de moneda a CLP numérico, estandarización de nombres, derivación de ATC anual, perfilado de columnas.
- Salida: Parquet canónico, ranking Top-N por menor ATC anual, endpoints de descarga y perfilado.

Fuera de alcance en Entrega 1:

- Parsing de documentos PDF.
- Datos personales o sensibles.

Fases siguientes:

- Incorporación de tablas del comparador de tarjetas SERNAC.
- Perfiles de uso paramétricos y recomendador.
- Persistencia relacional y endpoints públicos de ATC y recomendación.

COMPETENCIAS (niveles propuestos 1 a 5)

Estructurar información: nivel 5. Esquema canónico y validaciones a partir de datos reales.

Diseñar sistemas de datos: nivel 3. Parquet ahora, ORM y DB listos para la siguiente fase.

Técnicas de ML: nivel 3. Se deja para fases posteriores, es determinista.

Modelización estadística: nivel 3. Descriptivos, normalización a UF (Unidad de Fomento) y ranking.

Gestión de proyectos: nivel 3. Plan, riesgos y hitos definidos.

Cumplimiento legal y normativo: nivel 4. Solo fuentes públicas oficiales, con trazabilidad.

Visualización: nivel 3. Tabla comparativa simple y métricas de resumen.

DATOS

CMF simulador de cuentas vista, vista de simulación promedio.

Tipo: página HTML pública.

Campos observados en Parquet: Institución, Producto, Costo neto total (cadena con símbolo \$), source, extraction_date.

Uso: línea base para costo mensual y derivación de ATC anual.

UF oficial para normalización (elegir una de dos rutas, ambas válidas y oficiales):

a) CMF API v3 UF. Requiere API key y permite JSON o XML para día, mes o periodos.

b) Banco Central de Chile BDE GetSeries. Requiere credenciales; entrega series por código con rangos de fechas.

Campos observados en Parquet: fecha, valor_uf, source, serie_id, extraction_date.

SERNAC comparador de tarjetas de crédito.

Tipo: página HTML pública.

Estado: pendiente para fase 2, no necesario en E1.

NOTA SOBRE LOS DATASETS ACTUALES

cta_cuentavista_cmf.parquet

• Estructura recibida: Institución (texto), Producto (texto), Costo neto total (texto con \$ y separadores), Unnamed: 3 (nulo), source (texto), extraction_date (datetime).

• Limpieza mínima aplicada: eliminación de columna vacía, parseo de moneda a float en CLP, renombres canónicos (institucion, producto, cta_mensual_clp, fuente, fecha_captura) y derivación $cta_anual_clp = cta_mensual_clp \times 12$.

indicadores_uf.parquet

- Estructura recibida: fecha (datetime), valor_uf (float), source (texto), serie_id (texto), extraction_date (datetime).
- Limpieza mínima aplicada: renombres canónicos (fecha, valor, fuente, serie_id, fecha_captura).

RESULTADOS PRELIMINARES

Metodología breve:

- Los montos mensuales provienen de la simulación promedio CMF.
- ATC anual se calcula como ATC mensual multiplicado por 12.
- La UF se utilizará en la fase siguiente como normalizador, manteniendo fecha de referencia y trazabilidad de la serie.

Tabla 1. Top 10 cuentas vista más baratas por ATC mensual y anual (CLP)

N	Institución	Producto	CTA mensual (CLP)	CTA anual (CLP)
1	COOCRETAL	Cuenta Vista	0	0
2	ORIENCOOP	Cuenta Vista Ahorro Fácil	0	0
3	BANCO FALABELLA	Cuenta Vista (1)	400	4.800
4	BANCO SANTANDER	Cuenta Más Lucas	448	5.376
5	BANCO DE CHILE	Cuenta Fan	912	10.944
6	BANCO ESTADO	Cuenta Rut	1.800	21.600
7	SCOTIABANK	Cuenta Vista Individual	2.359	28.308
8	COOPEUCH	Cuenta Vista	2.775	33.300
9	BANCO ESTADO	Cuenta Pro (Chequera Electrónica)	3.925	47.100
10	BICE	Go UC	4.246	50.952

Tabla 2. Resumen estadístico del ATC mensual y anual en CLP

Métrica	CTA mensual (CLP)	CTA anual (CLP)
Mínimo	0	0
Percentil 25	912	10.944
Mediana	3.925	47.100
Percentil 75	7.076	84.912
Máximo	10.311	123.732
Promedio	3.983	47.792

Tabla 3. Resumen de la serie UF observada

Atributo	Valor
Rango de fechas	2024-12-02 a 2025-03-11
Valor UF mínimo	38.273,30
Valor UF máximo	38.794,07
Último valor del rango	38.794,07
Serie	F073.UFF.PRE.Z.D
Fuente	BDE_API

RIESGOS Y MITIGACIÓN

1. Cambios de estructura HTML en CMF o SERNAC. Mitigación: selectores robustos y pruebas. Si falla, CSV manual mínimo y trazable para continuidad.
2. Disponibilidad de servicios UF. Mitigación: alternar entre CMF API y BDE según disponibilidad y cachear últimas lecturas.
3. Frescura de datos. Mitigación: trazabilidad con fecha_captura y recapturas programadas.
4. Formatos numéricos heterogéneos. Mitigación: funciones centralizadas de parseo y validación de tipos.

PLANIFICACIÓN Y CRONOGRAMA. REVISAR

Hito 1 Limpieza y esquema canónico de CMF y UF. Estado: en curso.

Hito 2 EDA mínimo y ranking Top-N por ATC anual. Estado: pendiente inmediato.

Hito 3 Documento corto, checklist de calidad y referencias. Estado: pendiente inmediato.

Fase 2 SERNAC tarjetas, perfiles de uso y recomendaciones.

Fase 3 Persistencia relacional, endpoints públicos y tablero simple.

Fase	Actividad Principal	Hitos / Entregables	Fecha
1. Definición y MVP	Configuración y primeros scrapers.	Entrega 1 (Propuesta)	Dic 2025
2. Consolidación	Limpieza profunda y esquema canónico.	Esquema validado	Ene 2026
3. Persistencia	Migración a Base de Datos (PostgreSQL).	Entrega 2 (Avance DB)	Feb 2026
4. Expansión	Módulo SERNAC (Tarjetas).	Dataset Tarjetas	Mar 2026
5. Producto Final	API REST final y Dashboard.	API v1.0 Release	Abr 2026
6. Cierre	Escritura de memoria y defensa.	Entrega Final (TFB)	10 May 2026

MARCO TEÓRICO: BANK COST BENCHMARK API

Este capítulo profundiza en los fundamentos teóricos y tecnológicos que sustentan el desarrollo de la plataforma. Se abordan desde las arquitecturas de software moderno hasta los modelos matemáticos utilizados para la proyección de indicadores financieros.

Arquitectura de Software y Tecnologías Web

Para el desarrollo de aplicaciones web modernas, escalables y mantenibles, es crucial seleccionar una arquitectura que desacople la lógica de negocio de la interfaz de usuario.

Arquitectura Cliente-Servidor y REST API

El patrón de arquitectura **REST (Representational State Transfer)** define un conjunto de restricciones para crear servicios web. En este modelo, el **Backend** (servidor) expone recursos (datos) a través de endpoints HTTP estandarizados (GET, POST, etc.), y el **Frontend** (cliente) consume estos recursos para renderizar la interfaz. Esto permite que ambos componentes evolucionen de forma independiente.

Single Page Application (SPA)

Las **Single Page Applications (SPA)** aplicacion web que carga una sola página HTML y actualiza dinámicamente el contenido a medida que el usuario interactúa con la app, sin recargar la página completa.

- **React:** Biblioteca de JavaScript para construir interfaces de usuario. Utiliza un **DOM Virtual** para optimizar el renderizado y una arquitectura basada en **Componentes** reutilizables, lo que facilita el desarrollo modular.
- **Vite:** Herramienta de construcción (build tool), ofrece un entorno de desarrollo extremadamente rápido gracias al uso de módulos ES nativos en el navegador.

Microframeworks con Flask

Flask es un microframework para Python. A diferencia de frameworks "full-stack" como Django, Flask es ligero y no impone una estructura rígida ni dependencias innecesarias. Esto lo hace ideal para construir **APIs RESTful** eficientes y servicios de microarquitectura, permitiendo integrar fácilmente bibliotecas (como Pandas o Scikit-learn).

Contenerización y Orquestación con Docker

Más allá de la virtualización ligera, **Docker** se implementa en este proyecto para resolver la complejidad de un entorno políglota (Python y Node.js).

- **Aislamiento de Dependencias:** El backend de Machine Learning requiere librerías de C++ y versiones específicas de Python (para pmdarima y scikit-learn). Docker asegura que estas dependencias no entren en conflicto con el entorno de construcción del frontend.
- **Docker Compose para Multicontenedor:** Se utiliza para orquestar la comunicación entre el contenedor del **Backend (Flask)**, el contenedor del **Frontend (Vite/React)** y los volúmenes de datos, permitiendo levantar todo el ecosistema con un solo comando.
- **Escalabilidad Horizontal:** La arquitectura contenida permite que la API de benchmark pueda replicarse en múltiples instancias detrás de un balanceador de carga si el volumen de peticiones financieras aumenta.

Machine Learning y Predicción Financiera

El núcleo analítico de la plataforma se basa en la capacidad de predecir comportamientos futuros de indicadores económicos clave.

Series de Tiempo

Una serie de tiempo es una secuencia de puntos de datos indexados en orden cronológico. En finanzas, esto es fundamental para analizar la evolución de activos, acciones o indicadores macroeconómicos.

Modelo ARIMA

El modelo **ARIMA (AutoRegressive Integrated Moving Average)** es una de las técnicas más robustas para el pronóstico de series temporales.

- **AR (AutoRegressive):** Utiliza la relación entre una observación actual y observaciones previas.
- **I (Integrated):** Aplica diferenciación para hacer la serie estacionaria (eliminar tendencias).
- **MA (Moving Average):** Utiliza la dependencia entre una observación y el error residual de modelos previos.

Se utiliza la librería `pmdarima` para la selección automática de los hiperparámetros óptimos (p,d,q) del modelo mediante el algoritmo `auto_arima`, optimizando el criterio de información de Akaike (AIC).

p (Orden Autoregresivo - AR)

Representa el número de **lags** (observaciones pasadas) que el modelo utiliza para predecir el valor actual.

- Si $p=2$, el modelo asume que el valor de hoy depende directamente de lo que pasó ayer y anteayer. Se basa en la correlación entre los datos actuales y sus valores anteriores.

d (Grado de Diferenciación - I)

Indica el número de veces que se deben restar las observaciones pasadas a las actuales para que la serie de tiempo sea **estacionaria**.

- Una serie es estacionaria cuando sus propiedades estadísticas (media y varianza) son constantes en el tiempo. Si tus datos financieros tienen una tendencia alcista clara, el parámetro d se encarga de "eliminar" esa tendencia para que el modelo pueda trabajar con la variación real de los datos.

q (Orden de Media Móvil - MA)

Representa el tamaño de la ventana de **errores de pronóstico** anteriores que se incorporan al modelo.

- En lugar de mirar los valores pasados directamente (como hace p), q mira qué tan "equivocado" estuvo el modelo en los días anteriores. Ayuda a suavizar choques aleatorios o ruido en los datos financieros.

Métricas de Evaluación

- **MAPE (Mean Absolute Percentage Error):** Mide el tamaño del error en términos porcentuales. Un MAPE bajo (ej. $< 5\%$) indica un modelo altamente preciso y confiable para proyecciones de UF e IPC.

Contexto Económico y Financiero

La aplicación opera sobre un dominio específico: el mercado financiero chileno y sus indicadores regulatorios.

Indicadores Macroeconómicos

- **UF (Unidad de Fomento):** Unidad reajutable según la inflación chilena. Su predicción es crítica para la planificación de créditos hipotecarios y de consumo.
- **IPC (Índice de Precios al Consumidor):** Medida principal de la inflación interna que determina el valor diario de la UF.
- **Bonos del Tesoro (Treasury Yields):** Las tasas de los bonos de EE.UU. (10 años) actúan como tasa libre de riesgo global, influyendo en las tasas de interés locales.

Costo Total Anual (CTA)

El **CTA** es un indicador obligatorio en Chile que resume todos los costos asociados a un producto financiero (tasas, comisiones, seguros) en un solo porcentaje anual, facilitando la transparencia y comparación.

Inteligencia Artificial Generativa y LLMs

Large Language Models (LLMs)

Modelos de aprendizaje profundo (ej. Gemini) capaces de entender instrucciones complejas y generar texto coherente basado en vastos conjuntos de datos.

Generación Aumentada por Recuperación (RAG)

Para que el LLM entregue insights reales sobre el mercado chileno, se utiliza la técnica **RAG (Retrieval-Augmented Generation)**.

- Se "inyecta" información fresca (el valor de la UF del día o el ranking actual de costos de la API) en el contexto del prompt.
- Evitar "alucinaciones" del modelo y asegurar que los consejos financieros estén basados en los datos calculados por la plataforma en tiempo real.



REFERENCIAS

[Comisión para el Mercado Financiero. Simulador de cuentas de depósito a la vista: simulación promedio y definición de costo neto total. Accedido en diciembre de 2025.](#)

[Comisión para el Mercado Financiero. Documentación API CMF Bancos v3: Unidad de Fomento \(UF\). Accedido en diciembre de 2025.](#)

[Banco Central de Chile. Statistics Database API BDE: documentación GetSeries y catálogo de series. Accedido en diciembre de 2025.](#)

[Servicio Nacional del Consumidor. Comparador de tarjetas de crédito. Accedido en diciembre de 2025.](#)